sql.ipynb_

1.Explain the fundamental differences between DDL, DML, and DQL commands in SQL. Provide one example for each type of command.

- Data Definition Language (DDL) DDL commands are all about structure of your database. let you create, modify, or remove objects like tables, schemas, or indexes.
    - CREATE TABLE Students (ID INT PRIMARY KEY, Name VARCHAR(50));
- Data Manipulation Language (DML) DML commands deal with the actual data inside those structures. Once the blueprint exists, DML is what you use to insert new records, update existing ones, or delete data.
    - INSERT INTO Students (ID, Name) VALUES (1, 'Alice');
- Data Query Language (DQL) DQL is focused purely on retrieving information. It doesn't change the structure.
    - SELECT Name FROM Students WHERE ID = 1;

2.What is the purpose of SQL constraints? Name and describe three common types of constraints, providing a simple scenario where each would be useful

- Constraints are rules applied to columns or tables to ensure that the data stored is valid, consistent, and meaningful.
- 1.PRIMARY KEY Constraint
    - Purpose: Guarantees each row has a unique identifier and that this identifier cannot be NULL.
    - Scenario: In a Students table, StudentID is the primary key. This ensures no two students share the same ID and every student record has one
- 2.FOREIGN KEY Constraint
    - Purpose: Enforces referential integrity by linking a column in one table to the primary key of another.
    - Scenario: In an Enrollments table, StudentID is a foreign key referencing Students.StudentID. This ensures only valid students can be enrolled
- 3.CHECK Constraint
    - Purpose: Restricts values in a column based on a logical condition.
    - Scenario: In an Employees table, a Salary column might have CHECK (Salary > 0). This prevents invalid entries like negative salaries.
- 4.NOT NULL Constraint

    - Purpose: Ensures that a column cannot contain NULL values.
    - Scenario: In a Users table, the Email column is marked NOT NULL. This guarantees that every user record has an email address, preventing incomplete data
- 5.UNIQUE Constraint
    - Purpose: Ensures that all values in a column are distinct, though unlike the primary key, it allows NULL values.
    - Scenario: In a Customers table, the PhoneNumber column could be marked UNIQUE. This prevents two customers from registering with the same phone number, but still allows a record with no phone number if needed.

3.Explain the difference between LIMIT and OFFSET clauses in SQL. How would you use them together to retrieve the third page of results, assuming each page has 10 records?

- LIMIT
    - Specifies the maximum number of rows to return.
    - LIMIT 10 - returns only 10 rows.
- OFFSET
    - Skips a given number of rows before starting to return results.
    - OFFSET 20 - skips the first 20 rows, then starts returning rows
- SELECT * FROM table ORDER BY column LIMIT 10 OFFSET 20;

4.What is a Common Table Expression (CTE) in SQL, and what are its main benefits? Provide a simple SQL example demonstrating its usage.

- A Common Table Expression (CTE) is essentially a temporary, named result set that exists only during the execution of a single query. It's defined using the WITH clause and can be referenced just like a table or view within that query.
- WITH DeptAvg AS (SELECT Department, AVG(Salary) AS AvgSalary FROM Employees GROUP BY Department)
- SELECT e.Name, e.Department, e.Salary FROM Employees e JOIN DeptAvg d ON e.Department = d.Department WHERE e.Salary > d.AvgSalary;
- Benefits:
    - Clarity and readability: They make complicated queries easier to understand by replacing deeply nested subqueries with named sections.
    - Reusability: You can reference the same CTE multiple times in the main query, avoiding duplication.
    - Recursive capabilities: CTEs support recursion, which is especially useful for hierarchical data like organizational charts or category trees.

    - Modularity: They let you structure queries in stages, similar to how functions or modules work in programming.

Double-click (or enter) to edit

---

5. Describe the concept of SQL Normalization and its primary goals. Briefly explain the first three normal forms (1NF, 2NF, 3NF)

- Normalization in SQL is the process of organizing data in a relational database to reduce redundancy and improve data integrity. It involves structuring tables and relationships so that data is stored efficiently and consistently.
- Primary Goals of Normalization
    - Eliminate redundancy: Prevent duplicate data across tables.
    - Ensure data integrity: Reduce anomalies during insert, update, or delete operations.
    - Simplify maintenance: Make the database easier to manage and update.
    - Improve consistency: Ensure that data is logically stored and relationships are clear.
- First Normal Form (1NF)
    - Each column must contain atomic (indivisible) values.
    - No repeating groups or arrays.
- Second Normal Form (2NF)
    - Must already satisfy 1NF.
    - All non-key attributes must depend on the whole primary key, not just part of it.
    - Applies mainly to tables with composite keys.
- Third Normal Form (3NF)
    - Must already satisfy 2NF.
    - No transitive dependencies: non-key attributes should depend only on the primary key, not on other non-key attribute

6. Create a database named ECommerceDB and perform the following tasks: 1. Create the following tables with appropriate data types and constraints:

create database EcommerceDB;

use EcommerceDB;

-- Create Categories table

CREATE TABLE Categories (

```
    CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    CategoryName VARCHAR(50) NOT NULL UNIQUE
);

-- Create Products table
CREATE TABLE Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(100) NOT NULL UNIQUE,
    CategoryID INT,
    Price DECIMAL(10,2) NOT NULL,
    StockQuantity INT,
    FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);

-- Create Customers table
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY AUTO_INCREMENT,
    CustomerName VARCHAR(100) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    JoinDate DATE
);

-- Create Orders table
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY AUTO_INCREMENT,
    CustomerID INT,
    OrderDate DATE NOT NULL,
    TotalAmount DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

```sql
-- insertion
INSERT INTO Categories (CategoryID, CategoryName)
VALUES
(1, 'Electronics'),
(2, 'Books'),
(3, 'Home Goods'),
(4, 'Apparel');


-- insertion into products
INSERT INTO Products (ProductID, ProductName, CategoryID, Price, StockQuantity)
VALUES
(101, 'Laptop Pro', 1, 1200.00, 50),
(102, 'SQL Handbook', 2, 45.50, 200),
(103, 'Smart Speaker', 1, 99.99, 150),
(104, 'Coffee Maker', 3, 75.00, 80),
(105, 'Novel: The Great SQL', 2, 25.00, 120),
(106, 'Wireless Earbuds', 1, 150.00, 100),
(107, 'Blender X', 3, 120.00, 60),
(108, 'T-Shirt Casual', 4, 20.00, 300);


-- insertion into customers
INSERT INTO Customers (CustomerID, CustomerName, Email, JoinDate)
VALUES
(1, 'Alice Wonderland', 'alice@example.com', '2023-01-10'),
(2, 'Bob the Builder', 'bob@example.com', '2022-11-25'),
(3, 'Charlie Chaplin', 'charlie@example.com', '2023-03-01'),
(4, 'Diana Prince', 'diana@example.com', '2021-04-26');
```

-- insertion into orders

INSERT INTO Orders (OrderID, CustomerID, OrderDate, TotalAmount)

VALUES

(1001, 1, '2023-04-26', 1245.50),

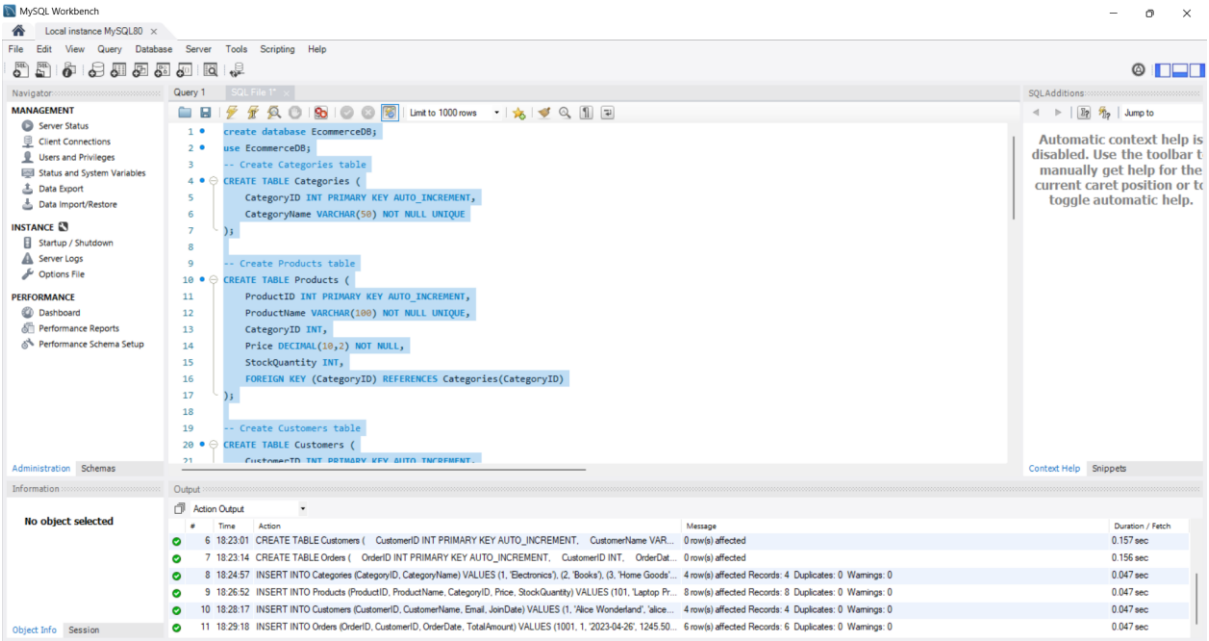
(1002, 2, '2023-10-12', 99.99),

(1003, 1, '2023-07-01', 145.00),

(1004, 3, '2023-01-14', 150.00),

(1005, 2, '2023-09-24', 120.00),

(1006, 1, '2023-06-19', 20.00);



7. Generate a report showing CustomerName, Email, and the TotalNumberofOrders for each customer. Include customers who have not placed any orders, in which case their TotalNumberofOrders should be 0. Order the results by CustomerName.

SELECT

c.CustomerName,

c.Email,

COUNT(o.OrderID) AS TotalNumberOfOrders

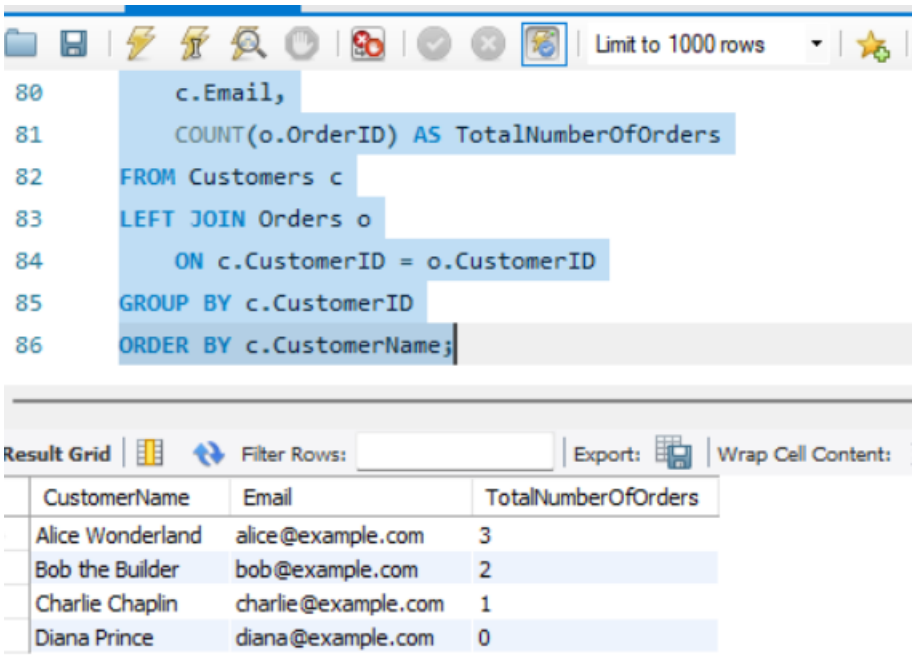
FROM Customers c

LEFT JOIN Orders o

ON c.CustomerID = o.CustomerID

GROUP BY c.CustomerID

ORDER BY c.CustomerName;



8. Retrieve Product Information with Category: Write a SQL query to display the ProductName, Price, StockQuantity, and CategoryName for all products. Order the results by CategoryName and then ProductName alphabetically.

SELECT

p.ProductName,

p.Price,

p.StockQuantity,

c.CategoryName

FROM Products p

JOIN Categories c

ON p.CategoryID = c.CategoryID

ORDER BY c.CategoryName, p.ProductName;

Query 1 | SQL File 1*

Limit to 1000 rows

```
    p.Price,
    p.StockQuantity,
    c.CategoryName
FROM Products p
JOIN Categories c
    ON p.CategoryID = c.CategoryID
ORDER BY c.CategoryName, p.ProductName;
```

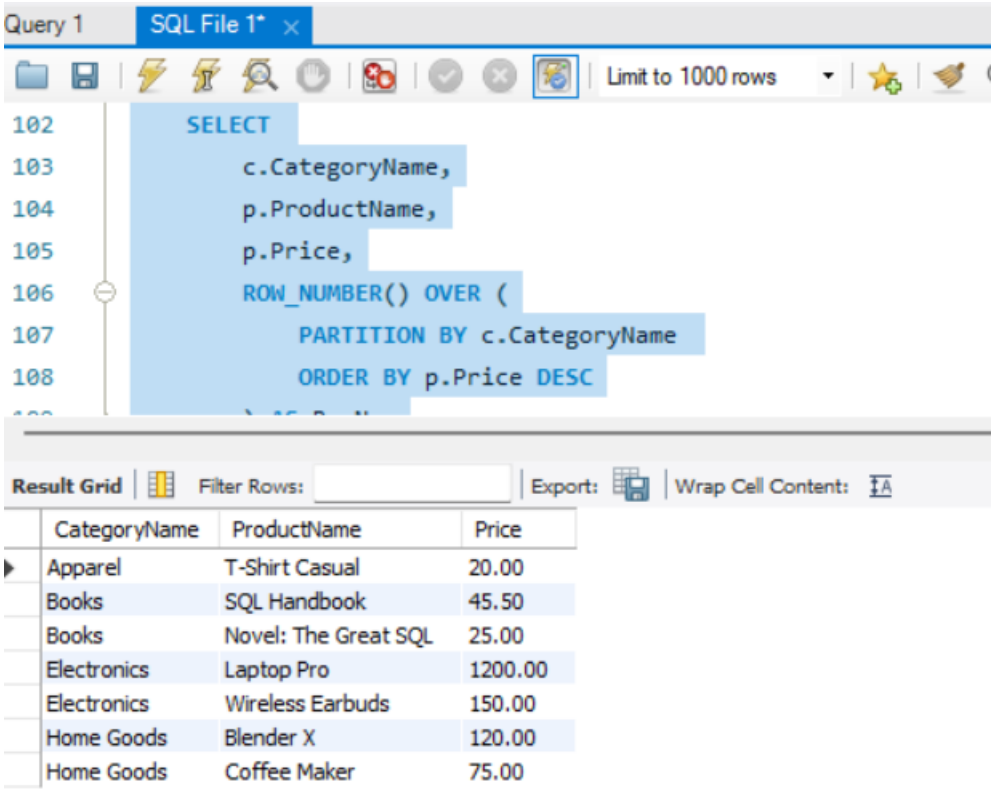
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ProductName	Price	StockQuantity	CategoryName
T-Shirt Casual	20.00	300	Apparel
Novel: The Great SQL	25.00	120	Books
SQL Handbook	45.50	200	Books
Laptop Pro	1200.00	50	Electronics
Smart Speaker	99.99	150	Electronics
Wireless Earbuds	150.00	100	Electronics
Blender X	120.00	60	Home Goods
Coffee Maker	75.00	80	Home Goods

9. write a SQL query that uses a Common Table Expression (CTE) and a Window Function (specifically ROW_NUMBER() or RANK()) to display the CategoryName, ProductName, and Price for the top 2 most expensive products in each CategoryName.

```
WITH RankedProducts AS (
  SELECT
    c.CategoryName,
    p.ProductName,
    p.Price,
    ROW_NUMBER() OVER (
      PARTITION BY c.CategoryName
      ORDER BY p.Price DESC
    ) AS RowNum
  FROM Products p
  JOIN Categories c
```

```
    ON p.CategoryID = c.CategoryID
)
SELECT
  CategoryName,
  ProductName,
  Price
FROM RankedProducts
WHERE RowNum <= 2
ORDER BY CategoryName, Price DESC;
```



CategoryName	ProductName	Price
Apparel	T-Shirt Casual	20.00
Books	SQL Handbook	45.50
Books	Novel: The Great SQL	25.00
Electronics	Laptop Pro	1200.00
Electronics	Wireless Earbuds	150.00
Home Goods	Blender X	120.00
Home Goods	Coffee Maker	75.00